

Travel Governance Pilot

Pre-Meeting Technical Brief

Connector × VeUP × Travel Client

Author: Umesh (Connector)

Date: April 2026

Status: **Early Stage** — **Pre-Discovery**

Classification: Confidential

Personal Note: This is my first read-through of what the client wants. I am writing this before the discovery call, so some of this is my interpretation from what was shared. I have a lot of open questions. My goal here is not to over-engineer before I know more — it is to walk in with a clear head, honest gaps, and a grounded technical direction. Connector has what they need at the core. The question is how much integration work sits on top of that.

What I Understood — 10 Points

This is my honest read of what the partner shared before any discovery call. It may be incomplete or partially wrong. I flag assumptions clearly.

1. **They have an AI-heavy travel web platform.** Their systems use AI for search ranking, booking recommendations, and CRM interactions. This is not a proof-of-concept AI setup — it is live, customer-facing, and generating real events.
2. **Their AI logs are disposable today.** Whatever the AI does — prompts, rankings, overrides — currently gets written to logs and either ignored or buried. There is no governance layer. This is the root problem.
3. **They have no audit trail they can show a regulator.** If someone asks “why did your AI recommend this package to this user?”, they cannot answer that today. That is a compliance exposure, not just a nice-to-have feature.
4. **They want a sidecar, not a replacement.** The framing of “embedded control plane alongside live systems” tells me they do not want to rip and replace. They want something that sits next to their existing stack and captures what is happening. That is the right instinct and it is the right pattern for Connector.
5. **Consent is a first-class requirement.** They specifically called out user consent handling. This means GDPR and CCPA are already on their radar. I assume they have some consent capture today but no enforcement layer.
6. **The PoC deliverable is deceptively simple to describe.** “Compile events into a `traveler_profile.md` and `trip_details.md`” sounds simple. It is not. Behind it is an ingestion pipeline, policy evaluation, tamper-evident storage, and a memory compilation step. The output file is the proof, not the product.
7. **“AWS-native” is a hard constraint.** They want everything in their own AWS account. This rules out SaaS governance tools (Fiddler, Truera, Arthur) and means Connector deploys as infrastructure in their environment. This is actually our advantage.
8. **This is a PoC, not production.** The goal is to prove the architecture works, not to handle 1M travellers. That said, every PoC architecture decision either helps or hurts the production path.
9. **Warm-start personalization is the business value.** The compliance angle is the hook. The real value they are selling internally is “AI that remembers context across sessions without asking users again.” Governance and personalization are the same pipeline — that framing is smart.
10. **The partner (VeUP) is handling client relationship and deployment.** My role is backend, core engine, and technical delivery. I need to scope what I can actually deliver in an 8-week PoC window given I am building and supporting solo.

Regulatory and Compliance Landscape

*What the regulatory environment actually looks like for a travel company using AI in 2025–2026. This shapes what the client **must** build, not just what they want to build.*

Key Regulations That Apply

Regulation	What It Covers	Risk If Ignored
GDPR (EU)	AI-driven personalization of EU travellers requires lawful basis, data minimization, and the right to explanation.	Fines up to €20M or 4% of global annual turnover. Travel companies with EU customers are directly exposed.
CCPA / CPRA (California)	User right to opt out of AI-driven profiling and targeted recommendations. Consent must be explicit and logged.	\$100–\$750 per consumer per incident. Class action risk is high for travel platforms with large US user bases.
US State Privacy Laws (20+ states, 2025)	Colorado, Virginia, Texas and others now have comprehensive AI profiling consent laws mirroring CCPA.	Patchwork enforcement is accelerating. No single compliance framework covers all states today.
EU AI Act (in force Aug 2025)	Customer-facing AI chatbots and recommendation engines must disclose AI involvement. High-risk classification if AI influences significant decisions.	Obligations for “limited risk” systems (chatbots, recommenders) active now. Non-compliance blocks EU market access.
IATA PNR Rules	Passenger Name Record data has strict cross-border data flow requirements. AI systems that process PNR must maintain audit logs for government access.	Airlines face fines up to USD \$10,000 per passenger for non-compliant data handling. Extends to OTA partners.
NIST AI RMF (US Voluntary, becoming standard)	Risk management framework for AI systems. Increasingly required by enterprise procurement.	Not legally binding yet, but B2B enterprise clients now include NIST AI RMF alignment in vendor requirements.

What This Means Practically

- Any AI system that ranks travel recommendations, adjusts prices, or personalises content **must** be able to explain its decisions on demand — to users and to regulators.
- Consent is not a checkbox. It must be **enforced at execution time**, logged with a timestamp and cryptographic proof, and honoured immediately on withdrawal.
- “We store logs in S3” is not a compliance answer. Regulators require structured, tamper-evident, replayable records — not raw text files.

- The EU AI Act's GPAI (General Purpose AI) obligations are live as of August 2025. Travel companies deploying third-party LLMs (GPT-4, Claude) are now considered **deployers** under the Act with direct obligations.

What They Need — 10 Points with Connector Fit

Each need is paired with a 2–3 line description of what Connector provides. This is honest: some things are fully solved, some need integration work.

What They Need	In Their Words	What Connector Provides
1. Real-time event capture	Capture prompts, outputs, rankings, human overrides as they happen across live systems.	POST /v1/capture endpoint with SDK wrappers for Python and Node.js. Events written to CID-addressed memory store in under 5ms.
2. Policy enforcement	Evaluate rules dynamically: consent checks, booking thresholds, PII restrictions — on every request.	Admission Gate runs a 5-layer guard pipeline (MAC → Policy → Content → Circuit Breaker → Audit) before any action executes. Deny-by-default, under 200 μ s overhead.
3. Consent management	Track user consent state per traveller and halt AI processing if consent is missing or withdrawn.	Consent state stored in structured namespace (/m/consent/{user_id}). Admission Gate checks consent on every capture event. Withdrawal triggers immediate enforcement.
4. Tamper-evident audit trail	Immutable, cryptographically verifiable log of what the AI did and why.	Every decision writes a EngineAuditEntry with a SHA-256 CID (soe1-sha256-*). Audit chain is append-only and signed with Ed25519. Regulator export supported.
5. Replay capability	Reconstruct exactly what the AI saw and decided at any past point in time.	SOE Surface Engine supports time-travel queries (--at, --since, --last). Returns a signed proof package with full decision context.
6. Memory compilation	Transform raw event stream into structured, reusable state objects (profiles, trips).	Memory Kernel compiles governed events into knowledge objects. Output: traveler_profile.md and trip_details.md with CID provenance chain.
7. Human-in-the-loop	High-value bookings require human review before AI proceeds.	Admission Gate auto-quarantines agents on policy violations and creates a HITL queue entry. Operator approves/denies via dashboard. Full audit log of human decision included.

What They Need	In Their Words	What Connector Provides
8. Observability dashboard	Operators need a real-time view of policies fired, consent states, and audit events.	Partial. SOE Surface Engine produces dashboard-ready surface documents via webhook. Grafana/Loki integration requires a thin adapter layer. PoC-scoped work.
9. AWS-native deployment	Everything must run in their AWS account. No external SaaS.	Connector is a self-hosted Rust binary. Deploys as ECS Fargate or EKS in their VPC. Terraform configs included. No data leaves their account.
10. Warm-start personalisation	AI should remember traveller context across sessions without re-collecting preferences.	Compiled memory objects persist in the Memory Kernel under <code>/m/traveller/{id}</code> . Any future AI call can query this as structured context. This is a direct output of the governance pipeline.

Architecture

PoC Architecture (8-Week Target)

The PoC proves one complete pipeline: **Event In** → **Policy Check** → **Audit** → **Memory Out**. It is intentionally narrow. We are not building production scale in the PoC.

PoC Stack — All in One ECS Fargate Container
Layer 5 (Visualise): Grafana dashboard, webhook-fed from Connector ↑ Webhook dispatcher pushes policy events + audit entries
Layer 4 (Control): Connector Platform (Rust binary) Admission Gate Surface Engine Policy Engine Memory Kernel
Layer 3 (Guardrails): 3 live travel policies (1) Consent check (2) Booking threshold \$5K → HITL (3) PII redaction
Layer 2 (Storage): redb (embedded) + S3 (audit export) CID-addressed memory Append-only audit chain Ed25519 signatures
Layer 1 (Capture): Python SDK + REST API Mock travel app → /v1/capture/search and /v1/capture/booking

Compute: Single ECS Fargate task, 1 vCPU / 2GB RAM, ~\$50/month.

Storage: 20GB EBS gp3 (hot) + S3 bucket (cold audit export).

Networking: ALB on port 8080, VPC-internal only. No public internet access required.

Deliverables at PoC end:

- Working capture pipeline with 3 enforced policies
- `traveler_profile.md` generated from governed event history
- `trip_details.md` with full audit chain reference
- Grafana dashboard showing live policy decisions
- Replay demo: given a date + user ID, produce a signed proof package
- Terraform config for one-click deploy in client's AWS account

Production Path (Post PoC)

Important: The PoC proves the pipeline. Production requires a different conversation about scale, multi-region, retention policy, and SLA. We should not promise production timelines in the first meeting.

When they grow beyond PoC, the natural path is:

Step 1. Horizontal scaling: Move from single ECS task to EKS cluster with horizontal pod autoscaling. Connector nodes run stateless; memory kernel is the only stateful component.

Step 2. Message queue: Replace synchronous HTTP capture with Kafka (AWS MSK). This decouples the travel app from the governance layer and handles traffic spikes without blocking.

- Step 3. Multi-region:** Deploy one Connector node per AWS region (us-east-1, eu-west-1, ap-southeast-1). Audit logs stay region-local for GDPR data residency. Cross-region policy sync via the license server.
- Step 4. Long-term retention:** redb for 30-day hot storage. S3 Standard for 1-year warm. S3 Glacier for 7-year compliance cold storage. Lifecycle policy auto-tiers.
- Step 5. Compliance reporting:** SOE Surface Engine exports structured compliance reports in PDF/JSON for regulators. GDPR “right to explanation” answered by the replay surface.

Limitations and Risks

Technical Limitations

- **No production Kafka integration yet.** The PoC uses synchronous HTTP capture. For high-throughput production, async ingestion via MSK needs to be built. This is 2–3 weeks of additional work.
- **Dashboard is webhook-fed, not native.** Grafana integration requires a thin webhook-to-OTLP adapter. It works but is not a polished product UI. A proper operator UI would be a Phase 2 deliverable.
- **No consent UI widget.** Connector enforces consent state but does not provide the front-end widget that collects it. The travel app must send consent state to Connector. We are the enforcement layer, not the collection layer.
- **Travel-specific policies are YAML-configured, not pre-built.** The 3 PoC policies need to be written and tested. They are straightforward but they require discovery input from the client (thresholds, field names, PII taxonomy).
- **Solo engineering.** I am building and supporting this. The 8-week timeline is achievable but assumes stable requirements. Scope creep is the biggest risk.

Risks

Risk	What Could Happen	Mitigation
Requirements shift	Client adds scope mid-PoC (“can you also do X?”). 8-week timeline breaks.	Scope freeze after Week 1. Any new requirement goes to Phase 2.
Access delays	AWS account access, API credentials, sample data take time to get.	Request access on Day 1. Use synthetic data for Weeks 1–2 while waiting.
Policy definition unclear	Client does not know exact field names, thresholds, or consent states in their system.	Run a 2-hour technical discovery session before Week 1 coding begins.
Travel app is complex	Their AI stack has many moving parts; capture points are unclear.	Start with 2 capture points only: search and booking. Expand later.
Compliance misalignment	Client’s legal team has different requirements than what engineering described.	Get a legal contact early. Document every policy decision in writing.

Maintenance Protocols (PoC Period)

- **Weekly sync:** 30-minute check-in with VeUP partner on progress, blockers, scope.
- **Incident response:** Critical issues (data loss, policy not enforcing) addressed within 4 hours during business days.
- **Policy updates:** YAML policy files can be hot-reloaded without restart. Non-breaking updates are self-service.

- **Audit exports:** Weekly S3 export of audit chain. Client can verify CID integrity independently.

Outcomes the Client Will Get

At the End of the PoC

- **A working governance sidecar deployed in their AWS account.** Not a demo environment — in their VPC, using their data.
- **Three enforced policies with live proof.** Every search and booking event passes through policy evaluation. Results logged and visible in Grafana.
- **A `traveler_profile.md` file for each synthetic test user.** Compiled from governed interaction history. Structured, versioned, CID-addressed.
- **A replay demo for a regulator scenario.** Given a user ID and a date, the system produces a signed proof package of exactly what the AI saw and decided. This is their answer to any regulatory inquiry.
- **Terraform infrastructure-as-code.** One command deploys the full stack. Repeatable. Auditable. Version-controlled.
- **A clear production roadmap.** What it costs, what it takes, and what the phased path looks like to move from PoC to 1M traveller scale.

Longer-Term Business Value

- **Compliance as a competitive moat.** They can tell enterprise clients and regulators they have an auditable AI system. Most competitors cannot.
- **Warm-start personalisation.** Traveller memory persists across sessions. Returning users get better recommendations immediately without re-collecting preferences.
- **Reduced legal risk.** GDPR right-to-explanation answered in under 5 seconds. CCPA opt-out enforced at the execution layer, not the log layer.

Questions — Technical and Commercial

Technical Questions I Need Answered Before Coding

- T1. What AI models are you using and where do they run?** OpenAI API, AWS Bedrock, a self-hosted model, or a mix? The capture integration differs for each.
- T2. What language/stack are your travel AI services written in?** Python, Node.js, Java? Determines which SDK I build first.
- T3. How are search queries and booking events structured today?** What fields exist in a typical event? Do you have a schema or data dictionary?
- T4. Where does user consent live today?** Is it in a CRM field, a database table, a cookie? Who is the source of truth for “user X has marketing consent”?
- T5. What counts as a “human override” in your system?** When a travel agent manually changes an AI recommendation, does that event exist anywhere today?
- T6. What does a high-value booking look like in your system?** Is \$5,000 the right threshold? Is it per booking, per day, per user?
- T7. Do you have any existing event streaming infrastructure?** Kafka, Kinesis, SQS, or are events only captured at the HTTP layer?
- T8. What PII fields does your AI system currently see?** Passport numbers, date of birth, loyalty IDs, credit card data? I need to know what to redact.
- T9. What is your current audit story?** CloudWatch logs? Nothing? An existing SIEM? Understanding the gap helps scope the delta.
- T10. What AWS services are already in use?** ECS, EKS, RDS, DynamoDB? I want to land in their existing patterns, not introduce new ones.

Commercial Questions for VeUP

- C1.** What is the client’s PoC budget? Is there a hard ceiling?
- C2.** What is the expected timeline from “yes” to first deployment? Do they have a hard date?
- C3.** Who owns any travel-specific policy logic we build? That is custom work on top of OSS.
- C4.** Is this PoC fixed-price or time-and-materials?
- C5.** What does “exclusive deployment partner” mean if the client eventually wants to buy Connector directly? I need to understand the commercial boundary before I start building.

Personal Open Questions

- P1.** The timeline of 8 weeks is achievable for the core pipeline. But if AWS access, sample data, and policy definition are delayed, we are building blind for weeks 1–2. How do we handle that?
- P2.** The PoC asks for a Grafana dashboard. Is this just a nice-to-have for the demo, or does the client have an ops team that will actually use it? That changes how much I invest in it.
- P3.** Are there any other AI systems beyond search and booking? CRM, pricing, fraud detection? Each one is a new capture point with its own policy needs.

This document is an early-stage working brief, not a proposal or commitment. All architecture decisions are subject to change after the discovery call. — Umesh, Connector